

~\Downloads\Day 23.sql

```
1  -- Task 1: Improve Customer Search Performance Using Search Optimization
2  CREATE OR REPLACE DATABASE ECOMMERCE_DB;
3
4  USE DATABASE ECOMMERCE_DB;
5  USE SCHEMA PUBLIC;
6
7
8  CREATE OR REPLACE TABLE CUSTOMERS
9  (
10     CUSTOMER_ID NUMBER,
11     CUSTOMER_NAME STRING,
12     EMAIL STRING,
13     PHONE STRING,
14     CITY STRING
15 );
16
17
18 INSERT INTO CUSTOMERS VALUES
19 (1,'Aarav Sharma','aarav@gmail.com','9876543210','Mumbai'),
20 (2,'Ananya Singh','ananya@gmail.com','9988776655','Delhi'),
21 (3,'Rahul Verma','rahul@gmail.com','9123456789','Pune'),
22 (4,'Priya Patel','priya@gmail.com','9871122334','Ahmedabad'),
23 (5,'John David','john@gmail.com','9874455667','Hyderabad'),
24 (6,'Alice Brown','alice@gmail.com','9011223344','Chennai'),
25 (7,'David Wilson','david@gmail.com','8888999900','Bangalore'),
26 (8,'Akash Mehta','akash@gmail.com','9877001122','Mumbai'),
27 (9,'Sneha Kapoor','sneha@gmail.com','9090909090','Delhi'),
28 (10,'Rohan Shah','rohan@gmail.com','9879988776','Surat'),
29 (11,'Ankit Jain','ankit@gmail.com','8765432109','Jaipur'),
30 (12,'Aditi Joshi','aditi@gmail.com','9871111222','Nagpur'),
31 (13,'Kiran Kumar','kiran@gmail.com','9012345678','Lucknow'),
32 (14,'Nisha Gupta','nisha@gmail.com','9872233445','Indore'),
33 (15,'Arjun Rao','arjun@gmail.com','9875566778','Mysore');
34
35
36 SELECT *
37 FROM CUSTOMERS
38 WHERE CUSTOMER_NAME LIKE 'A%';
39
40
41 SELECT *
42 FROM CUSTOMERS
43 WHERE EMAIL='john@gmail.com';
44
45 SELECT *
46 FROM CUSTOMERS
47 WHERE PHONE LIKE '%987%';
48
```

```
49  EXPLAIN
50
51  SELECT *
52
53  FROM CUSTOMERS
54
55  WHERE CUSTOMER_NAME LIKE 'A%';
56
57  EXPLAIN
58
59  SELECT *
60
61  FROM CUSTOMERS
62
63  WHERE PHONE LIKE '%987%';
64
65  ALTER TABLE CUSTOMERS
66  ADD SEARCH OPTIMIZATION;
67
68  ALTER TABLE CUSTOMERS
69  ADD SEARCH OPTIMIZATION
70  ON SUBSTRING(PHONE);
71
72  SELECT *
73  FROM CUSTOMERS
74  WHERE CUSTOMER_NAME LIKE 'A%';
75
76  EXPLAIN
77
78  SELECT *
79
80  FROM CUSTOMERS
81
82  WHERE PHONE LIKE '%987%';
83
84
85  -- Expected Observations
86  -- Which queries improved?
87  -- Email lookup (=) improves significantly.
88  -- Prefix searches (LIKE 'A%') improve.
89  -- Substring search (LIKE '%987%') improves after enabling substring optimization
90
91
92
93
94
95  -- Task 2: Cortex AI
96  CREATE OR REPLACE TABLE CUSTOMER_REVIEWS
97  (
98      REVIEW_ID NUMBER,
```

```
99     CUSTOMER_NAME STRING,
100     REVIEW_TEXT STRING
101 );
102
103
104 INSERT INTO CUSTOMER_REVIEWS VALUES
105 (1,'Rahul','Excellent shopping experience. Fast delivery and great quality. '),
106 (2,'Priya','Very bad packaging. Product arrived damaged. '),
107 (3,'John','The product is okay. Nothing special. '),
108 (4,'Alice','Amazing customer support. '),
109 (5,'David','Late delivery but product quality is good. '),
110 (6,'Sneha','Worst experience ever. '),
111 (7,'Akash','Highly recommend this seller. '),
112 (8,'Nisha','Average quality for the price. '),
113 (9,'Aditi','Loved the product. '),
114 (10,'Arjun','Could be improved with better packaging. ');
115
116
117 SELECT
118
119 REVIEW_ID,
120
121 CUSTOMER_NAME,
122
123 REVIEW_TEXT,
124
125 SNOWFLAKE.CORTEX.SENTIMENT(REVIEW_TEXT) AS SENTIMENT
126
127 FROM CUSTOMER_REVIEWS;
128
129
130 SELECT
131
132 SNOWFLAKE.CORTEX.SUMMARIZE
133 (
134 'The delivery was delayed by five days. Although the product quality was excellent, the
135 packaging was damaged and customer support took two days to respond. Overall I am satisfied
136 but the delivery process should improve.'
137 )
138 AS SUMMARY;
139
140
141 SELECT
142
143 SNOWFLAKE.CORTEX.TRANSLATE
144 (
145 'The product quality is excellent.',
146 'English',
147 'French'
148 );
```

```
147
148 SELECT
149
150 AI_COMPLETE
151 (
152 'Suggest three improvements for an online shopping platform.'
153 );
154
155 SELECT *
156
157 FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY;
158
159
160 -- Which Cortex function consumed the most credits?
161 /* SELECT
162 FUNCTION_NAME,
163 SUM(CREDITS_USED) AS TOTAL_CREDITS
164 FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY
165 GROUP BY FUNCTION_NAME
166 ORDER BY TOTAL_CREDITS DESC
167 LIMIT 1; */
168
169
170 -- Which model consumed the highest credits?
171 /* SELECT
172 MODEL_NAME,
173 SUM(CREDITS_USED) AS TOTAL_CREDITS
174 FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY
175 GROUP BY MODEL_NAME
176 ORDER BY TOTAL_CREDITS DESC
177 LIMIT 1; */
178
179
180 --Which user executed the most AI functions?
181 /* SELECT
182 USER_NAME,
183 COUNT(*) AS TOTAL_REQUESTS
184 FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY
185 GROUP BY USER_NAME
186 ORDER BY TOTAL_REQUESTS DESC
187 LIMIT 1; */
```